Complete Technical Documentation

WASP-52 Twin-Sine Photometric Variability Pipeline with MCMC

`phot_var_twin_sine_QC_v12_mcmc.py` & `v10_compare_runs_to_overleaf_mcmc.py`

Abstract

This document walks through *every function* in both pipeline scripts, in the chronological order they actually execute, grouped into four core stages – **Quality Control**, **GLS Period Search**, **Single & Twin-Sine Model**, and **MCMC Posterior & Statistical Treatment** – plus the supporting data-ingestion utilities, pipeline orchestration (`main()`), and the comparison/Overleaf-report generator. Every subsection gives the governing equation(s), the exact source code, and the file + line numbers where it lives, so any result can be traced back to the code that produced it.

Contents

1 Pipeline Overview

The analysis script's `main()` (phot_var_twin_sine_QC_v12_mcmc.py, line 1085) executes the following call sequence – this is the chronological backbone the rest of this document follows:

1. `read_table()` → `auto_map_columns()` → `prepare_data()` – ingest raw CSV, map columns, convert mag→flux if needed (Section 2).
2. `smart_clean()` (internally: `tag_sessions()`) – multi-level outlier rejection (Section 3).
3. `tag_sessions()` → `summarize_sessions()` → `flag_bad_sessions()` → `rollup_observers()` – post-cleaning QC diagnostics/reporting (Section 3).
4. `prewhiten_twin_sine()` – internally calls `gls_periodogram()`, `apply_exclude_notches()`, `fit_weighted_sine()`, `fit_twin_sine()`, `chi2_metrics()`, and the nested `estimate_period_err()` (Sections 4 and 5).
5. `run_all_mcmc()` (opt-in via `-mcmc`) – internally calls `run_mcmc_fit()` four times and `make_mcmc_plots()` (Section 6).
6. `make_plots()` – all diagnostic and phase-folded figures (Section 5, phase folding).

The second script, v10_compare_runs_to_overleaf_mcmc.py, is run afterward, independently, to aggregate one or more output directories from step 6 into a single LaTeX/PDF comparison report (Section 8).

2 Utilities & Data Ingestion

2.1 `fig_save_all_ext` / `plt_save_all_ext`

Function: `fig_save_all_ext, plt_save_all_ext` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 49-59`

```python
def fig_save_all_ext(fig, path_with_ext: str, *, dpi_png: int = 200) -> None:
    """Save a Matplotlib Figure to both .png and .pdf using the same basename."""
    base, _ = os.path.splitext(path_with_ext)
    fig.savefig(base + ".png", dpi=dpi_png, bbox_inches="tight")
    fig.savefig(base + ".pdf", dpi=dpi_png, bbox_inches="tight")

def plt_save_all_ext(path_with_ext: str, *, dpi_png: int = 200) -> None:
    """Like fig_save_all_ext but uses the current pyplot figure."""
    base, _ = os.path.splitext(path_with_ext)
    plt.savefig(base + ".png", dpi=dpi_png, bbox_inches="tight")
    plt.savefig(base + ".pdf", dpi=dpi_png, bbox_inches="tight")
```

No mathematics – pure I/O helpers so every figure is saved as both a raster (PNG, for quick viewing) and a vector (PDF, for publication) copy.

2.2 `timestamp` / `ensure_dir`

Function: `timestamp, ensure_dir` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 61-65`

```python
def timestamp():
    return datetime.now().strftime("%Y%m%d_%H%M%S")

def ensure_dir(path):
    Path(path).mkdir(parents=True, exist_ok=True)
```

Bookkeeping utilities; no mathematics.

2.3 `mad` – Median Absolute Deviation

Function: `mad` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 67-71`

$$\mathrm{MAD}(x) = 1.4826 \cdot \mathrm{median}\left(|x_i - \mathrm{median}(x)|\right)$$

```
SCALE_MAD = 1.4826

def mad(x, c=SCALE_MAD):
    x = np.asarray(x, float)
    return c * np.nanmedian(np.abs(x - np.nanmedian(x)))
```

A robust, outlier-insensitive alternative to the standard deviation. The constant $1.4826 = 1/\Phi^{-1}(0.75)$ makes it a consistent estimator of σ for Gaussian data. This is the single most-reused building block in the pipeline – it underlies every sigma-clipping rule in Section 3.

2.4 `weighted_stats`

Function: `weighted_stats` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 73-79`

$$\bar{y} = \frac{\sum_i w_i y_i}{\sum_i w_i}, \qquad s = \sqrt{\frac{\sum_i w_i (y_i - \bar{y})^2}{\sum_i w_i}}$$

```
def weighted_stats(y, w):
    wsum = np.sum(w)
    if wsum == 0:
        return np.nan, np.nan
    mu = np.sum(w * y) / wsum
    var = np.sum(w * (y - mu)**2) / wsum
    return mu, np.sqrt(var)
```

Inverse-variance-weighted mean/std ($w_i = 1/\sigma_i^2$). Used only to seed the amplitude starting guess before nonlinear fitting (Section 5).

2.5 `read_table` / `auto_map_columns`

Function: `read_table, auto_map_columns` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 93-130`

```
def read_table(path, delimiter=None, comment=None, skiprows=None):
    kw = {}
    if comment not in (None, ""):
        kw["comment"] = comment
    if isinstance(skiprows, int):
        kw["skiprows"] = skiprows
    try_delims = [delimiter] if delimiter else [",","\t",";"]
    for delim in try_delims:
        try:
            df = pd.read_csv(path, delimiter=delim, engine="python", **kw)
            if df.shape[1] > 1:
                return df
        except Exception:
            pass
    df = pd.read_csv(path, delim_whitespace=True, engine="python", **kw)
    return df

def auto_map_columns(df, user_map=None):
    colmap = {}
    cols_lower = {c.lower(): c for c in df.columns}
    if user_map:
```

```
        for k,v in user_map.items():
            if v in df.columns:
                colmap[k] = v
    for key, cands in DEFAULT_COLS.items():
        if key in colmap:
            continue
        for cand in cands:
            if cand in df.columns:
                colmap[key] = cand; break
            if cand.lower() in cols_lower:
                colmap[key] = cols_lower[cand.lower()]; break
    if "time" not in colmap or ("flux" not in colmap and "mag" not in colmap):
        raise ValueError(
            "Could not find required columns for time/(flux or magnitude). "
            f"Found: {df.columns.tolist()} Map: {colmap}"
        )
    return colmap
```

No mathematics – `read_table` auto-detects the delimiter (comma/tab/semicolon/whitespace); `auto_map_columns` fuzzy-matches raw CSV headers (e.g. `"Observer Code"`, `JD`, `Magnitude`) against a table of known aliases (`DEFAULT_COLS`) so the same code accepts both AAVSO raw exports and pre-cleaned CSVs.

2.6 `prepare_data` – unit conversion & normalization

Function: `prepare_data` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 132-170`

Magnitude→flux (Pogson relation):

$$f_i = 10^{-0.4\,(m_i - \tilde{m})}, \qquad \tilde{m} = \text{median}(m)$$

Propagated error (from $df/dm = -0.4\ln(10)\,f$):

$$\sigma_{f,i} = |0.4 \ln 10|\, f_i\, \delta m_i$$

Normalization:

$$f_i \to \frac{f_i}{\text{median}(f)}, \qquad \sigma_{f,i} \to \frac{\sigma_{f,i}}{\text{median}(f)}$$

```
def prepare_data(df, colmap, normalize=True):
    out = pd.DataFrame()
    out["time"] = df[colmap["time"]].astype(float).values

    if "flux" in colmap:
        out["flux"] = df[colmap["flux"]].astype(float).values
        if "flux_err" in colmap:
            out["flux_err"] = df[colmap["flux_err"]].astype(float).values
        else:
            sig = mad(out["flux"].values); out["flux_err"] = np.full(len(out), sig if np.isfinite(sig) else
                  1.0)
        out.attrs["flux_mode"] = "flux"

    elif "mag" in colmap:
        mvals = df[colmap["mag"]].astype(float).values
        dm = df[colmap["mag_err"]].astype(float).values if "mag_err" in colmap else np.full(len(mvals), np.
             nan)
        m_med = np.nanmedian(mvals)
        rel_flux = 10.0**(-0.4 * (mvals - m_med))
        k = 0.4*np.log(10.0)
        rel_flux_err = np.where(np.isfinite(dm), np.abs(k)*rel_flux*dm, np.nan)
        out["flux"] = rel_flux
        if np.all(~np.isfinite(rel_flux_err)):
            sig = mad(rel_flux); rel_flux_err = np.full(len(rel_flux), sig if np.isfinite(sig) else 1.0)
        out["flux_err"] = rel_flux_err
```

```
        out.attrs["flux_mode"] = "mag->flux"
    else:
        raise ValueError("Neither flux nor mag columns were found after mapping.")

    out["observer"] = df[colmap.get("observer","observer")].astype(str).values if colmap.get("observer")
         else "unknown"
    out["filter"] = df[colmap.get("filt","filter")].astype(str).values if colmap.get("filt") else "unknown
        "
    out["flag"] = df[colmap.get("flag","flag")].values if colmap.get("flag") else 0

    m = np.isfinite(out["time"]) & np.isfinite(out["flux"]) & np.isfinite(out["flux_err"])
    out = out[m].copy()

    if normalize:
        med = np.nanmedian(out["flux"])
        if np.isfinite(med) and med != 0:
            out["flux"] /= med; out["flux_err"] /= med
    return out
```

If no error column exists (or all propagated errors are non-finite), `mad()` of the flux itself is substituted as a constant fallback uncertainty for every point.

2.7 `parse_exclude_periods`

Function: `parse_exclude_periods` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 1070-1083`

```
def parse_exclude_periods(s):
    if not s:
        return None
    out = []
    for chunk in str(s).split(";"):
        chunk = chunk.strip()
        if not chunk:
            continue
        if "-" in chunk:
            lo,hi = chunk.split("-",1)
            lo=float(lo.strip()); hi=float(hi.strip())
            if hi<lo: lo,hi = hi,lo
            out.append((lo,hi))
    return out if out else None
```

Parses the `-exclude-periods` CLI string (e.g. `"0.95-1.05;1.60-1.90"`) into a list of (lo, hi) tuples consumed by `apply_exclude_notches()` (Section 4). No statistical content.

3 Quality Control

3.1 `tag_sessions` – session/night definition

Function: `tag_sessions` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 173-198` **Gap-based:**

$$s_i = \sum_{j \le i} \mathbb{1}\left[\Delta t_j > \frac{\text{gap\_hours}}{24}\right], \qquad \Delta t_j = t_j - t_{j-1}$$

Calendar-based (astronomical night):

$$\text{night}_i = \lfloor t_i + 0.5 \rfloor$$

```
def tag_sessions(df: pd.DataFrame, group: str = "gap", gap_hours: float = 6.0) -> pd.DataFrame:
    """Add 'session_id' (e.g., OBS#S3) and 'night_id' columns."""
    d = df.sort_values(["observer", "time"]).copy()
```

```
    if group not in {"gap", "calendar"}:
        raise ValueError("--group must be 'gap' or 'calendar'")

    sess_ids = []
    night_ids = []
    for obs, g in d.groupby("observer", sort=False):
        tt = g["time"].to_numpy()
        if group == "gap":
            gap_days = gap_hours / 24.0
            edges = np.r_[True, np.diff(tt) > gap_days] # new session if big gap
            sess_num = np.cumsum(edges) # 1..K per observer
            night_id = sess_num
        else:
            night_id = np.floor(tt + 0.5).astype(int) # astronomical night
            _, sess_num = np.unique(night_id, return_inverse=True)
            sess_num += 1

        sess_ids.extend([f"{obs}#S{n}" for n in sess_num])
        night_ids.extend(night_id)

    d["session_id"] = sess_ids
    d["night_id"] = night_ids
    return d
```

Gap-based: a new session starts whenever consecutive points (per observer, sorted in time) are separated by more than `gap_hours`. *Calendar-based*: JD increments at noon UT, so +0.5 before flooring shifts the boundary to local midnight. Called **twice** in `main()`: once pre-fit for QC diagnostics (line 1261) and once on the cleaned data before fitting (line 1308).

3.2 `per_group_metrics` – per-session quality metrics

Function: `per_group_metrics` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 200-227`

χ^2-proxy (a per-session *mean*, not dof-normalized):

$$\chi^2_{\text{proxy}} = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{r_i}{\sigma_i}\right)^2, \qquad r_i = f_i - \text{median}(f)$$

Outlier fraction:

$$\text{ofrac} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}\Big[\,|r_i - \text{median}(r)| > 4 \cdot \max\big(\text{MAD}(r),\, \text{std}(r),\, 1\big)\,\Big]$$

```
def per_group_metrics(g: pd.DataFrame) -> dict:
    N = len(g)
    med_err = float(np.nanmedian(g["flux_err"]))
    rmad = float(mad(g["resid"].values))
    mean = float(np.nanmean(g["resid"].values))
    std = float(np.nanstd(g["resid"].values))

    # chi2/dof proxy with robust fallback
    if np.isfinite(g["flux_err"]).any():
        denom = g["flux_err"].to_numpy(copy=True)
        pos = denom[np.isfinite(denom) & (denom > 0)]
        mpos = np.nanmedian(pos) if pos.size else np.nan
        if np.isfinite(mpos):
            denom[~np.isfinite(denom) | (denom <= 0)] = mpos
        chi = float(np.nanmean((g["resid"].to_numpy() / denom) ** 2))
    else:
        chi = np.nan
```

```
    medr = float(np.nanmedian(g["resid"]))
    thresh = 4.0 * (rmad if (np.isfinite(rmad) and rmad > 0) else (std if std > 0 else 1.0))
    ofrac = float(np.mean(np.abs(g["resid"].to_numpy() - medr) > thresh))

    return dict(
        N=N, med_err=med_err,
        resid_mean=mean, resid_std=std, resid_MAD=rmad,
        chi2_over_dof=chi, outlier_frac=ofrac,
        t_min=float(np.nanmin(g["time"])), t_max=float(np.nanmax(g["time"]))
    )
```

Non-finite/non-positive errors are patched with the session's own median positive error before dividing, so a handful of bad uncertainties cannot blow up the statistic.

3.3 `summarize_sessions`

Function: `summarize_sessions` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 229-241`

```
def summarize_sessions(df: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for (obs, sess), g in df.groupby(["observer", "session_id"], sort=False):
        m = per_group_metrics(g)
        m.update(
            observer=str(obs),
            session_id=str(sess),
            night_id=str(int(np.nanmedian(g["night_id"])) if np.isfinite(np.nanmedian(g["night_id"])) else
                -1),
        )
        rows.append(m)
    s = pd.DataFrame(rows)
    sort_cols = ["chi2_over_dof", "resid_MAD", "outlier_frac", "N"]
    return s.sort_values(sort_cols, ascending=[False, False, False, False]).reset_index(drop=True)
```

Applies `per_group_metrics()` to every (observer, session) group and sorts worst-first. No new mathematics.

3.4 `rollup_observers`

Function: `rollup_observers` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 243-252`

```
def rollup_observers(session_df: pd.DataFrame) -> pd.DataFrame:
    g = session_df.groupby("observer", as_index=False)
    obs = g.agg(
        N_total=("N", "sum"),
        n_sess=("session_id", "count"),
        med_MAD=("resid_MAD", "median"),
        med_chi=("chi2_over_dof", "median"),
        med_ofrac=("outlier_frac", "median"),
    )
    return obs
```

Simple groupby-aggregate (sum/count/median) rolling session-level metrics up to the observer level.

3.5 `flag_bad_sessions` – bad-session criterion

Function: `flag_bad_sessions` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 254-265`

$$\text{bad(session)} = \big(N \geq N_{\min}\big) \;\wedge\; \Big[\big(\chi^2_{\text{proxy}} \geq \tau_\chi \text{ or MAD} > f_{\text{MAD}} \cdot \widetilde{\text{MAD}}\big) \vee \big(\text{ofrac} \geq \tau_o\big)\Big]$$

```
def flag_bad_sessions(sessions: pd.DataFrame, chi_thresh: float, ofrac_thresh: float, minN: int,
     mad_factor: float) -> pd.Series:
    """ Mark a session as bad if it has enough points AND (chi2/dof high OR outlier_frac high).
        If chi2/dof is unavailable (all NaN), fall back to a robust MAD-based criterion. """
    enough_points = sessions["N"] >= minN
    use_mad = sessions["chi2_over_dof"].isna().all()
    if use_mad:
        ref = np.nanmedian(sessions["resid_MAD"])
        chi_or_mad_bad = sessions["resid_MAD"] > (mad_factor * ref)
    else:
        chi_or_mad_bad = sessions["chi2_over_dof"] >= chi_thresh
    ofrac_bad = sessions["outlier_frac"] >= ofrac_thresh
    return enough_points & (chi_or_mad_bad | ofrac_bad)
```

If the χ^2-proxy is unavailable for every session (all NaN), a MAD-ratio fallback substitutes for it. `main()` additionally computes *observer*-level exclusion as

$$\text{exclude(observer)} = \left(\frac{\#\text{bad sessions}}{\#\text{sessions}}\right) \geq f_{\text{bad,obs}}$$

directly in `main()` (phot_var_twin_sine_QC_v12_mcmc.py, line 918: `obs["exclude_observer"] = obs["frac_bad_sessions"] >= args.obs_bad_frac`), not inside a separate function.

3.6 `_barh` – QC bar-chart helper

Function: `_barh` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 267-281`

```
def _barh(figfile, labels, values, title, xlabel, height_per=0.35, min_h=6, fontsize=8, xlim=None):
    fig_h = max(min_h, height_per * len(labels) + 1.5)
    fig, ax = plt.subplots(figsize=(10, fig_h))
    y = np.arange(len(labels))[::-1]
    ax.barh(y, values, alpha=0.85)
    ax.set_yticks(y, labels)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.grid(axis="x", alpha=0.3)
    ax.tick_params(axis="y", labelsize=fontsize)
    if xlim is not None:
        ax.set_xlim(*xlim)
    fig.tight_layout()
    fig_save_all_ext(fig, figfile, dpi_png=160)
    plt.close(fig)
```

Plotting utility for the observer/session quality bar charts (`07_*`, `08_*` output figures). No statistical content.

3.7 `smart_clean` – the multi-stage cleaning pipeline

Function: `smart_clean` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 284-403` All three sigma-clip stages share the generic rule:

$$\text{reject } y_i \text{ if } |y_i - \text{median}(y)| > k \cdot \text{MAD}(y)$$

Stage 2 (per-session normalize + clip): $f_i \to f_i/\text{median}(f_{\text{session}})$, then reject if $|f_i - m_0| > \sigma_{\text{night}} \cdot \text{MAD}(f_{\text{session}})$. **Stage 3 (global iterative clip):** repeated `global_iters` times, recomputing median/MAD each pass, reject if $|f_i - \mu| > \sigma_{\text{global}} \cdot \text{MAD}$. **Stage 4 (rolling-window clip):** for window $\mathcal{W}_i = \{j : |t_j - t_i| \leq h\}$, $h = \text{roll\_window}/2$: reject if $|f_i - \text{median}(f_{\mathcal{W}_i})| > \sigma_{\text{roll}} \cdot \text{MAD}(f_{\mathcal{W}_i})$.

```python
def smart_clean(df: pd.DataFrame, *,
                flag_reject_values=None,
                err_max=None,
                err_median_factor=None,
                group="gap", gap_hours=6.0,
                per_night_minN=20, per_night_sigma=3.0,
                global_sigma=3.0, global_iters=2,
                roll_window=0.5, # full width [days]; 0 disables
                roll_sigma=4.0,
                thin_maxN=0,
                outdir=None):
    """ Returns: cleaned DataFrame, removed DataFrame (+ reason).

    Steps:
    0. optional: reject flags, gate by error (absolute or relative to median)
    1. session tagging (observer, astronomical night or gap-based)
    2. per-session normalization (divide by session median) + within-session MAD clip
    3. global robust sigma clip (iterations)
    4. optional rolling-window MAD clip
    5. optional thinning (logged as 'thinned_for_runtime')
    """
    work = df.copy()
    removed_records = []

    # 0) flags & error gates
    if flag_reject_values is not None and "flag" in work.columns:
        bad = work["flag"].isin(flag_reject_values)
        if np.any(bad):
            r = work.loc[bad].copy(); r["reason"] = "flag_reject"
            removed_records.append(r); work = work.loc[~bad]
    if err_median_factor is not None:
        cap = err_median_factor * np.nanmedian(work["flux_err"].values)
        err_max = cap if err_max is None else min(err_max, cap)
    if err_max is not None:
        bad = work["flux_err"] > err_max
        if np.any(bad):
            r = work.loc[bad].copy(); r["reason"] = f"flux_err>{err_max}"
            removed_records.append(r); work = work.loc[~bad]

    # 1) sessions
    work = tag_sessions(work, group=group, gap_hours=gap_hours)

    # 2) per-session normalization + within-session clip
    blocks = []
    for (obs, sess), g in work.groupby(["observer", "session_id"], sort=False):
        gg = g.copy()
        if len(gg) >= per_night_minN:
            med = np.nanmedian(gg["flux"])
            if np.isfinite(med) and med > 0:
                gg["flux"] = gg["flux"] / med
                gg["flux_err"] = gg["flux_err"] / med
            m0 = np.nanmedian(gg["flux"])
            s0 = mad(gg["flux"].values)
            if np.isfinite(s0) and s0 > 0:
                bad = np.abs(gg["flux"] - m0) > per_night_sigma * s0
                if np.any(bad):
                    r = gg.loc[bad].copy(); r["reason"] = f"per_night_sigma({obs},{sess})"
                    removed_records.append(r); gg = gg.loc[~bad]
        blocks.append(gg)
    work = pd.concat(blocks, ignore_index=True)

    # 3) global robust sigma-clip (iterated)
    for k in range(int(global_iters)):
        mu = np.nanmedian(work["flux"])
        s = mad(work["flux"].values)
        if not np.isfinite(s) or s == 0:
            break
```

```
        bad = np.abs(work["flux"] - mu) > global_sigma * s
        if np.any(bad):
            r = work.loc[bad].copy(); r["reason"] = f"global_sigma_iter{k+1}"
            removed_records.append(r); work = work.loc[~bad]

    # 4) rolling-window MAD clip
    if roll_window and len(work) > 0:
        w = work.sort_values("time").copy()
        half = 0.5 * roll_window
        keeper = np.ones(len(w), bool)
        t = w["time"].to_numpy()
        f = w["flux"].to_numpy()
        for i, ti in enumerate(t):
            m = (t >= ti - half) & (t <= ti + half)
            mu = np.nanmedian(f[m])
            sg = mad(f[m])
            if np.isfinite(sg) and sg > 0 and np.abs(f[i]-mu) > roll_sigma * sg:
                keeper[i] = False
        if not np.all(keeper):
            r = w.loc[~keeper].copy(); r["reason"] = f"rolling_{roll_sigma}sigma"
            removed_records.append(r)
        work = w.loc[keeper].copy()

    # 5) thinning
    if thin_maxN and len(work) > thin_maxN:
        step = max(1, len(work) // thin_maxN)
        thinned_idx = work.index[1::step] # keep 0, drop every 'step'th
        r = work.loc[thinned_idx].copy(); r["reason"] = "thinned_for_runtime"
        removed_records.append(r)
        work = work.drop(index=thinned_idx).reset_index(drop=True)

    removed = (pd.concat(removed_records, ignore_index=True)
               if removed_records else pd.DataFrame(columns=list(df.columns)+["reason"]))

    # summaries
    if outdir:
        if len(removed) > 0:
            removed.to_csv(os.path.join(outdir, "removed_points.csv"), index=False)
        obs_summary = work.groupby("observer").size().rename("kept").to_frame()
        rem_summary = (removed.groupby("observer").size().rename("removed").to_frame()
                       if len(removed) else pd.DataFrame())
        summary = obs_summary.join(rem_summary, how="outer").fillna(0).astype(int)
        summary.to_csv(os.path.join(outdir, "cleaning_summary_by_observer.csv"))
        sess_summary = work.groupby(["observer","session_id"]).size().rename("kept").reset_index()
        sess_summary.to_csv(os.path.join(outdir, "cleaning_summary_by_session.csv"), index=False)
        with open(os.path.join(outdir,"cleaning_report.txt"),"w") as f:
            f.write("CLEANING REPORT\n")
            f.write(f"Total input points: {len(df)}\n")
            f.write(f"Total kept: {len(work)}\n")
            f.write(f"Total removed: {len(removed)}\n")

    return work.reset_index(drop=True), removed
```

This is the single largest QC function – five sequential stages (flag/error gating, per-session clip, global iterative clip, rolling-window clip, optional thinning), each logging removed points with a `reason` string for full traceability in `removed_points.csv`.

4 GLS Period Search

4.1 `gls_periodogram` – Generalized Lomb–Scargle

Function: `gls_periodogram` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 406-418`

$$y(t) \approx A\cos(\omega t) + B\sin(\omega t) + C, \qquad \mathcal{P}(\omega) = \frac{\chi_0^2 - \chi^2(\omega)}{\chi_0^2}, \qquad \omega = \frac{2\pi}{P}$$

$$f_{\min} = \frac{1}{P_{\max}},\ f_{\max} = \frac{1}{P_{\min}},\ f_k = f_{\min} + k\Delta f,\ P_k = \frac{1}{f_k}$$

```
def gls_periodogram(time, flux, flux_err, minP, maxP, nfreq=100000):
    if not (np.isfinite(minP) and np.isfinite(maxP)) or (minP>=maxP):
        return np.array([]), np.array([]), None
    t = np.asarray(time,float); y = np.asarray(flux,float); e = np.asarray(flux_err,float)
    if ASTROPY_OK:
        ls = LombScargle(t,y,e); fmin = 1.0/maxP; fmax = 1.0/minP
        f = np.linspace(fmin, fmax, nfreq); power = ls.power(f); P = 1.0/f
        return P, power, ls
    else:
        y = y - np.average(y, weights=1/np.maximum(e,1e-12)**2)
        f = np.linspace(1.0/maxP, 1.0/minP, nfreq); ang = 2*np.pi*f
        power = lombscargle(t, y, ang); P = 1.0/f
        return P, power, None
```

Uses `astropy.timeseries.LombScargle` (Zechmeister & Kürster 2009 generalized/floating-mean formulation) when available, falling back to `scipy.signal.lombscargle` on a manually mean-subtracted series otherwise. The frequency grid is linear in frequency (not period), scanning from $1/P_{\max}$ to $1/P_{\min}$.

4.2 `_safe_gls_window`

Function: `_safe_gls_window` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 420-423`

```
def _safe_gls_window(time,y,yerr,lo,hi,nfreq):
    if (lo is None) or (hi is None) or (not np.isfinite(lo)) or (not np.isfinite(hi)) or (lo>=hi):
        return np.array([]), np.array([]), None
    return gls_periodogram(time,y,yerr,lo,hi,nfreq)
```

Thin guard wrapper around `gls_periodogram()` for the narrow harmonic-search windows (Section 4.4), returning empty arrays instead of raising if the window is degenerate.

4.3 `apply_exclude_notches`

Function: `apply_exclude_notches` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 425-432`

$$\mathcal{P}(P) \to -\infty \quad \text{for } P \in [\text{lo}, \text{hi}]$$

```
def apply_exclude_notches(Pgrid, power, exclude_ranges):
    if not exclude_ranges:
        return power
    pow2 = power.copy()
    for (lo,hi) in exclude_ranges:
        mask = (Pgrid>=lo) & (Pgrid<=hi)
        pow2[mask] = -np.inf
    return pow2
```

Masks user-specified nuisance period ranges (e.g. 1-day aliases, via `-exclude-periods`) so they can never be selected as the periodogram peak.

4.4 `prewhiten_twin_sine` – period-search portion

Function: `prewhiten_twin_sine` **File:** `phot_var_twin_sine_QC_v12_mcmc.py`, lines 838-964 (period-search logic: 838-897)

Prior window (if `-P1-prior` given):

$$[\mathrm{lo}_{\mathrm{eff}}, \mathrm{hi}_{\mathrm{eff}}] = \Big[\max(P_{\min}, P_{\mathrm{prior}}(1-\epsilon)),\ \min(P_{\max}, P_{\mathrm{prior}}(1+\epsilon))\Big]$$

Prewhitened residual (subtract fitted fundamental before searching for P_2):

$$r_i^{(1)} = f_i - \big[C_1 + A_1 \sin(2\pi t_i/P_1 + \phi_1)\big]$$

Harmonic search window: $P_{\mathrm{half}} = 0.5P_1$, $P_{\mathrm{double}} = 2P_1$,

$$\mathrm{window}(P_c) = \big[\max(P_{\min}, P_c(1-w)),\ \min(P_{\max}, P_c(1+w))\big]$$

```
def prewhiten_twin_sine(time, flux, flux_err, minP, maxP, nfreq=100000,
                        harmonic_window=0.15,
                        P1_fixed=None,P1_prior=None,P1_prior_frac=0.25,
                        exclude_ranges=None, ic_mode="chi2"):
    t = np.asarray(time,float)

    # prior window
    lo_eff,hi_eff = minP,maxP
    if P1_prior is not None:
        lo_eff = max(minP, P1_prior*(1.0-P1_prior_frac))
        hi_eff = min(maxP, P1_prior*(1.0+P1_prior_frac))
        if lo_eff>=hi_eff:
            lo_eff,hi_eff = minP,maxP

    # choose P1
    if P1_fixed is not None:
        P1=float(P1_fixed); Pgrid=np.array([P1]); power=np.array([1.0])
    else:
        Pgrid,power,_ = gls_periodogram(time,flux,flux_err,lo_eff,hi_eff,nfreq)
        if len(Pgrid)==0:
            raise ValueError("Global GLS returned no frequencies. Check minP/maxP/prior.")
        power_use = apply_exclude_notches(Pgrid,power,exclude_ranges)
        i1 = int(np.nanargmax(power_use)); P1=float(Pgrid[i1])

    # fundamental fit
    (A1_tmp,phi1_tmp,C1_tmp),_ = fit_weighted_sine(time,flux,flux_err,P1)
    model_single = C1_tmp + sine_model(time,A1_tmp,phi1_tmp,P1)
    resid1 = flux - model_single
    metrics_single = chi2_metrics(flux, flux_err, model_single, k_params=3, mode=ic_mode)

    # harmonic search
    P_half = 0.5*P1; P_double = 2.0*P1
    def window(Pc):
        lo=max(minP, Pc*(1.0-harmonic_window)); hi=min(maxP, Pc*(1.0+harmonic_window))
        return lo,hi
    lo_h,hi_h = window(P_half); lo_d,hi_d = window(P_double)
    Ph,pow_h,_ = _safe_gls_window(time,resid1,flux_err,lo_h,hi_h,max(nfreq//4,5000))
    Pd,pow_d,_ = _safe_gls_window(time,resid1,flux_err,lo_d,hi_d,max(nfreq//4,5000))

    P2=None; harmonic_chosen=""
    P2_h = Ph[np.argmax(pow_h)] if len(Ph) else None
    P2_d = Pd[np.argmax(pow_d)] if len(Pd) else None
    if len(Ph) and len(Pd):
        if np.max(pow_h) >= np.max(pow_d):
            P2=P2_h; harmonic_chosen="0.5*Prot"
        else:
            P2=P2_d; harmonic_chosen="2*Prot"
```

```
    elif len(Ph):
        P2=P2_h; harmonic_chosen="0.5*Prot"
    elif len(Pd):
        P2=P2_d; harmonic_chosen="2*Prot"
    else:
        Pglob,powglob,_ = gls_periodogram(time,flux,flux_err,minP,maxP,nfreq)
        for idx in np.argsort(powglob)[::-1]:
            if np.abs(Pglob[idx]-P1)/P1 > 0.05:
                P2=Pglob[idx]; harmonic_chosen="fallback_global"; break
        if P2 is None:
            candidate=0.5*P1; candidate = min(max(candidate,minP),maxP)
            P2=candidate; harmonic_chosen="forced_valid_range"
```

P_1 is the global periodogram peak (optionally restricted to a prior window / fixed by the user). The fundamental sine is fit and subtracted ("prewhitened"), then a *second* periodogram is run on the residual, restricted to narrow windows around $0.5P_1$ and $2P_1$ – motivated by the physical picture of two spot groups producing signal at the rotation period and its harmonic. Whichever window's peak is higher is adopted as P_2, with global-search and forced-fallback branches if neither window yields a peak. (The remainder of this function – the twin-sine fit itself – is documented in Section 5.5.)

4.5 `estimate_period_err` (nested inside `prewhiten_twin_sine`)

Function: `estimate_period_err` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 908-947 (nested function; called at lines 951, 954)` **Parabolic fit:** $\mathcal{P}(P) \approx aP^2+bP+c$. **$1/e$ half-width** (if $a < 0$, a genuine peak):

$$aP^2 + bP + c = \frac{P_{\max}}{e} \;\Rightarrow\; \sigma_P = \frac{|P_+ - P_-|}{2}$$

FWHM fallback (if $a \geq 0$):

$$\mathrm{FWHM} = 2\sqrt{2\ln 2}\,\sigma \;\Rightarrow\; \sigma_P = \frac{\mathrm{FWHM}}{2.355}$$

```
    def estimate_period_err(Pgrid, power, Ppeak):
        """Estimate period uncertainty from periodogram peak width."""
        if len(Pgrid) < 5:
            return np.nan
        for frac in [0.02, 0.05, 0.10, 0.15]:
            m = (Pgrid > Ppeak * (1 - frac)) & (Pgrid < Ppeak * (1 + frac))
            if not np.any(m) or np.sum(m) < 5:
                continue
            Pg, pw = Pgrid[m], power[m]
            try:
                a, b, c = np.polyfit(Pg, pw, 2)
                if a >= 0:
                    pmax = np.max(pw)
                    target = pmax / 2.0
                    above_half = pw > target
                    if np.sum(above_half) >= 2:
                        indices = np.where(above_half)[0]
                        fwhm = Pg[indices[-1]] - Pg[indices[0]]
                        return fwhm / 2.355
                    continue
                pmax = np.max(pw)
                target = pmax / np.e
                roots = np.roots([a, b, c - target])
                roots = roots[np.isreal(roots)].real
                if len(roots) == 2:
                    return 0.5 * np.abs(roots[1] - roots[0])
            except Exception:
                continue
        return np.nan
```

Retried over progressively wider windows (2%, 5%, 10%, 15% around the peak) until enough points are available. Called twice in `prewhiten_twin_sine` (lines 951, 954) to get `P1_err_est` and `P2_err_est`, which later seed the MCMC free-period Gaussian prior width (Section 6).

5 Single & Twin-Sine Model

5.1 `sine_model` / `twin_sine_model`

Function: `sine_model, twin_sine_model` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 434-438`

$$y(t) = C + A\sin\left(\frac{2\pi t}{P} + \phi\right) \qquad y(t) = C + A_1\sin\left(\frac{2\pi t}{P_1} + \phi_1\right) + A_2\sin\left(\frac{2\pi t}{P_2} + \phi_2\right)$$

```
def sine_model(t,A,phi,P):
    return A*np.sin(2*np.pi*t/P + phi)

def twin_sine_model(t,A1,phi1,P1,A2,phi2,P2,C):
    return C + sine_model(t,A1,phi1,P1) + sine_model(t,A2,phi2,P2)
```

A = semi-amplitude, ϕ = phase offset, P = period, C = baseline flux. The twin-sine model is the pipeline's core physical model: rotational modulation at P_1 plus a secondary signal at its harmonic/sub-harmonic P_2.

5.2 `fit_weighted_sine` / `fit_twin_sine`

Function: `fit_weighted_sine, fit_twin_sine` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 440-458`

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \sum_i \frac{\left(y_i - \hat{y}(t_i;\theta)\right)^2}{\sigma_i^2}$$

```
def fit_weighted_sine(t,y,yerr,P,y0=None):
    if y0 is None:
        y0=np.median(y)
    w = 1.0/np.maximum(yerr,1e-12)**2
    mu,sig = weighted_stats(y-y0,w); A0 = sig if (np.isfinite(sig) and sig>0) else 0.01
    def f(tt,A,phi,C):
        return C + sine_model(tt,A,phi,P)
    popt,pcov = curve_fit(f,t,y,sigma=yerr,p0=[A0,0.0,y0],absolute_sigma=True,maxfev=20000)
    return popt,pcov

def fit_twin_sine(t,y,yerr,P1,P2,y0=None):
    if y0 is None:
        y0=np.median(y)
    w = 1.0/np.maximum(yerr,1e-12)**2
    mu,sig = weighted_stats(y-y0,w); A0 = sig if (np.isfinite(sig) and sig>0) else 0.01
    def f(tt,A1,phi1,A2,phi2,C):
        return twin_sine_model(tt,A1,phi1,P1,A2,phi2,P2,C)
    popt,pcov = curve_fit(f,t,y,sigma=yerr,p0=[A0,0.0,0.5*A0,0.0,y0],absolute_sigma=True,maxfev=40000)
    return popt,pcov
```

Both periods are held *fixed* at the periodogram values (Section 4); only amplitude(s), phase(s), and the baseline are free. Starting guess A_0 comes from `weighted_stats()` (Section 2). `absolute_sigma=True` means the returned covariance assumes `yerr` is already properly calibrated.

5.3 `gaussian_nll` – Gaussian negative log-likelihood

Function: `gaussian_nll` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 460-467`

$$\mathrm{NLL} = \frac{1}{2}\sum_{i=1}^{n}\left[\frac{(y_i-\hat{y}_i)^2}{\sigma_i^2} + \ln(2\pi\sigma_i^2)\right]$$

```
def gaussian_nll(y, yerr, ymodel):
    """
    Gaussian negative log-likelihood (NLL) for independent points with known errors.
    NLL = 0.5 * sum( (r^2 / var) + ln(2*pi*var) ), where var = yerr^2 (floored).
    """
    resid = np.asarray(y, float) - np.asarray(ymodel, float)
    var = np.maximum(np.asarray(yerr, float), 1e-12)**2
    return 0.5 * np.sum(resid**2 / var + np.log(2.0 * np.pi * var))
```

The exact NLL under independent-Gaussian errors. Unlike plain χ^2, it also penalizes/rewards the width of the error bars via the $\ln(2\pi\sigma_i^2)$ term. **This exact function is reused, unmodified, as the MCMC log-likelihood** in Section 6.

5.4 `chi2_metrics` – goodness of fit & information criteria

Function: `chi2_metrics` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 469-524`

$$\chi^2 = \sum_{i=1}^{n}\frac{(y_i-\hat{y}_i)^2}{\sigma_i^2}, \quad \mathrm{dof} = n-k, \quad \chi^2_r = \frac{\chi^2}{\max(\mathrm{dof},1)}$$

$$\mathrm{AIC}_\chi = \chi^2 + 2k, \quad \mathrm{BIC}_\chi = \chi^2 + k\ln n \qquad \mathrm{AIC}_{\ell\ell} = 2k + 2\,\mathrm{NLL}, \quad \mathrm{BIC}_{\ell\ell} = k\ln n + 2\,\mathrm{NLL}$$

```
def chi2_metrics(y, yerr, ymodel, k_params, mode="chi2"):
    y = np.asarray(y, float); yerr = np.asarray(yerr, float); ymodel = np.asarray(ymodel, float)
    resid = y - ymodel
    var = np.maximum(yerr, 1e-12)**2

    chi2 = np.sum(resid**2 / var)
    n = len(y)
    dof = n - k_params
    rchi2 = chi2 / max(dof, 1)

    aic_chi = chi2 + 2 * k_params
    bic_chi = chi2 + k_params * np.log(n)

    nll = 0.5 * np.sum(resid**2 / var + np.log(2.0 * np.pi * var))
    aic_ll = 2 * k_params + 2 * nll
    bic_ll = k_params * np.log(n) + 2 * nll

    out = dict(
        chi2=chi2, dof=dof, rchi2=rchi2,
        AIC=aic_chi, BIC=bic_chi,
        nll=nll, AIC_ll=aic_ll, BIC_ll=bic_ll,
        AIC_chi=aic_chi, BIC_chi=bic_chi,
    )
    if mode == "gauss":
        out["AIC"] = aic_ll
        out["BIC"] = bic_ll
    return out
```

Both IC conventions are *always* computed and stored; `-ic-mode` only selects which pair populates the headline `AIC`/`BIC` keys. Called with k=3 for the single-sine model and k=5 for the twin-sine model, so a genuinely lower AIC/BIC for the twin model is real evidence for a second periodicity, not just a byproduct of extra free parameters.

5.5 `prewhiten_twin_sine` – model-fitting portion

Function: `prewhiten_twin_sine` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 899-964`

$$T_{\text{span}} = \max(t) - \min(t), \qquad N_{\text{cycles}} = \frac{T_{\text{span}}}{P_1}$$

```
    # twin-sine fit
    (A1,phi1,A2,phi2,C),cov = fit_twin_sine(time,flux,flux_err,P1,P2)
    model_twin = twin_sine_model(time,A1,phi1,P1,A2,phi2,P2,C)
    perr = np.sqrt(np.diag(cov)) if cov is not None else [np.nan]*5

    params = dict(A1=A1,A1_err=perr[0],phi1=phi1,phi1_err=perr[1],P1=P1,
                  A2=A2,A2_err=perr[2],phi2=phi2,phi2_err=perr[3],P2=P2,
                  C=C,C_err=perr[4],harmonic_choice=harmonic_chosen)

    if ASTROPY_OK and (P1_fixed is None):
        Pglob,powglob,_ = gls_periodogram(time,flux,flux_err,lo_eff,hi_eff,nfreq)
        sigP1 = estimate_period_err(Pglob,powglob,P1)
        resid_for_P2 = flux - (C + sine_model(time,A1,phi1,P1))
        Ph2,powh2,_ = gls_periodogram(time,resid_for_P2,flux_err,max(minP,0.25*P1),min(maxP,4*P1),max(nfreq
            //2,5000))
        sigP2 = estimate_period_err(Ph2,powh2,P2)
    else:
        sigP1=np.nan; sigP2=np.nan
    params["P1_err_est"]=sigP1; params["P2_err_est"]=sigP2
    tspan = np.max(t)-np.min(t)
    params["Tspan_days"]=tspan; params["cycles_at_P1_over_span"]=(tspan/P1 if (np.isfinite(P1) and P1>0)
         else np.nan)

    metrics_twin = chi2_metrics(flux, flux_err, model_twin, k_params=5, mode=ic_mode)

    return (Pgrid,power,P1,resid1,Ph,pow_h,Pd,pow_d,P2,params,
           model_single,metrics_single,model_twin,metrics_twin)
```

Parameter uncertainties $\sigma_{\theta_k} = \sqrt{\Sigma_{kk}}$ come from the diagonal of `fit_twin_sine`'s covariance matrix (the linearized/Gauss–Newton approximation `curve_fit` returns) – these are the *pre-MCMC* "preliminary" error bars referenced in Section 6. `cycles_at_P1_over_span` is a data-quality sanity check: a period claim spanning very few rotation cycles is far less trustworthy than one covering dozens.

5.6 `make_plots` – phase-folding portion

Function: `make_plots` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 967-1067 (phase-folding logic: lines 1030-1067)`

$$t_{\text{ref}} = -\frac{\phi_1 P_1}{2\pi}, \qquad \varphi_i = \left(\frac{t_i - t_{\text{ref}}}{P_1}\right) \bmod 1, \quad \varphi_i \to \varphi_i - 1 \text{ if } \varphi_i > 0.5$$

```
    # Phase-folded at P1 (zero-centered, annotate P1)
    tref = - (param_dict["phi1"] * P1) / (2*np.pi)
    phi = ((time - tref) / P1) % 1.0
    phi[phi > 0.5] -= 1.0

    plt.figure(figsize=(7,5))
    plt.errorbar(phi, flux, yerr=flux_err, fmt=".", ms=2, alpha=0.6, label="data")
    ph_grid = np.linspace(-0.5, 0.5, 1000)
    t_grid = tref + ph_grid * P1
    y_med = np.nanmedian(flux)
    vert_shift = y_med - param_dict["C"]
    m1 = (param_dict["C"] + vert_shift) + sine_model(t_grid, param_dict["A1"], param_dict["phi1"], P1)
```

```
    plt.plot(ph_grid, m1, lw=1.6, ls="--", label=f"fundamental only (P1={P1:.3f} d)")
    ...
    save_all(os.path.join(outdir,"06_phase_P1.png")); plt.close()

    # Full-model phase plot (06b)
    m_full = vert_shift + twin_sine_model(
        t_grid, param_dict["A1"], param_dict["phi1"], P1,
        param_dict["A2"], param_dict["phi2"], P2, param_dict["C"]
    )
    ...
    save_all(os.path.join(outdir, "06b_phase_P1_fullmodel.png")); plt.close()
```

t_{ref} aligns phase 0 with the fitted sine's own phase reference; wrapping to $[-0.5, 0.5)$ keeps a symmetric feature centered rather than split across plot edges. `vert_shift` is a purely cosmetic constant shift so the plotted model visually sits on the data's median – it never touches the fitted parameters. `make_plots` also produces the six other diagnostic figures (`01_raw_cleaned` through `05_residuals`), which are direct visualizations of already-computed quantities with no new equations.

6 MCMC Posterior & Statistical Treatment

This entire section is additive: it reuses `sine_model`, `twin_sine_model`, and `gaussian_nll` from Section 5 verbatim as the likelihood, and only adds a posterior-sampling layer around the *same* equations, run only if `-mcmc` is passed. It supports four combinations: $(\text{model\_type}, \text{period\_mode}) \in \{\text{single}, \text{twin}\} \times \{\text{fixed}, \text{free}\}$.

6.1 MCMC_PARAM_SPECS

Function: `MCMC_PARAM_SPECS` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 549-555`

```
MCMC_PARAM_SPECS = {
    ("single", "fixed"): [("A", "amp"), ("phi", "phase"), ("C", "const")],
    ("single", "free"): [("A", "amp"), ("phi", "phase"), ("C", "const"), ("P", "period")],
    ("twin", "fixed"): [("A1", "amp"), ("phi1", "phase"), ("A2", "amp"), ("phi2", "phase"), ("C", "const")
        ],
    ("twin", "free"): [("A1", "amp"), ("phi1", "phase"), ("A2", "amp"), ("phi2", "phase"),
                       ("C", "const"), ("P1", "period"), ("P2", "period")],
}
```

A declarative parameter-vector layout for each of the four combinations, letting a single set of generic functions (below) handle all four without duplicated code.

6.2 _mcmc_build_model

Function: `_mcmc_build_model` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 557-570`

```
def _mcmc_build_model(theta, spec, fixed, t):
    vals = dict(zip([s[0] for s in spec], theta))
    vals.update(fixed) # fixed periods (only present when period_mode == "fixed")
    is_twin = "A2" in vals
    if is_twin:
        return twin_sine_model(t, vals["A1"], vals["phi1"], vals["P1"],
                               vals["A2"], vals["phi2"], vals["P2"], vals["C"])
    else:
        return vals["C"] + sine_model(t, vals["A"], vals["phi"], vals["P"])
```

Dispatches an MCMC parameter vector θ to the exact same `sine_model`/`twin_sine_model` functions from Section 5 – no new physics, just a different caller.

6.3 _wrap_phase

Function: `_wrap_phase` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 572-575`

$$\varphi_{\text{wrapped}} = (\varphi + \pi) \bmod 2\pi - \pi$$

```
def _wrap_phase(x):
    return (np.asarray(x, float) + np.pi) % (2 * np.pi) - np.pi
```

Wraps phase(s) to $[-\pi, \pi]$ for *reporting only* – never affects the sampler, since $\sin(2\pi t/P + \phi)$ is already exactly periodic in ϕ.

6.4 _mcmc_log_prior

Function: `_mcmc_log_prior` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 577-603`

$$\text{amp} \sim \text{Unif}(0, A_{\max}) \qquad \text{const} \sim \text{Unif}(C_{\text{lo}}, C_{\text{hi}}) \qquad \text{phase} : \text{flat/unbounded}$$

$$\text{period} \sim \mathcal{N}(P_0, \sigma_P) \text{ truncated to } [P_{\min}, P_{\max}], \qquad \ln \mathcal{N} = -\tfrac{1}{2}\left(\tfrac{P-P_0}{\sigma_P}\right)^2 - \ln(\sigma_P\sqrt{2\pi})$$

```
def _mcmc_log_prior(theta, spec, prior_cfg):
    lp = 0.0
    for (name, kind), val in zip(spec, theta):
        if kind == "amp":
            lo, hi = prior_cfg["amp_bounds"]
            if not (lo <= val <= hi):
                return -np.inf
        elif kind == "const":
            lo, hi = prior_cfg["const_bounds"]
            if not (lo <= val <= hi):
                return -np.inf
        elif kind == "period":
            P0, sigmaP, lo, hi = prior_cfg["period_priors"][name]
            if not (lo <= val <= hi):
                return -np.inf
            lp += -0.5 * ((val - P0) / sigmaP) ** 2 - np.log(sigmaP * np.sqrt(2 * np.pi))
        # kind == "phase": flat/unbounded, contributes 0
    return lp
```

$A \geq 0$ breaks the $(A \to -A,\ \phi \to \phi+\pi)$ degeneracy. The period prior is centered on the periodogram peak with width set from `estimate_period_err()` (Section 4.4), so the free-period MCMC explores around the already-identified period/harmonic rather than an unrelated alias.

6.5 _mcmc_log_likelihood / _mcmc_log_posterior

Function: `_mcmc_log_likelihood, _mcmc_log_posterior` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 605-621`

$$\ln \mathcal{L}(\theta) = -\text{NLL}(\theta) \qquad \ln P(\theta \mid y) \ \propto\ \ln \pi(\theta) + \ln \mathcal{L}(\theta)$$

```
def _mcmc_log_likelihood(theta, spec, fixed, t, y, yerr):
    ymodel = _mcmc_build_model(theta, spec, fixed, t)
    return -gaussian_nll(y, yerr, ymodel)

def _mcmc_log_posterior(theta, spec, fixed, prior_cfg, t, y, yerr):
    lp = _mcmc_log_prior(theta, spec, prior_cfg)
    if not np.isfinite(lp):
        return -np.inf
    ll = _mcmc_log_likelihood(theta, spec, fixed, t, y, yerr)
    if not np.isfinite(ll):
        return -np.inf
    return lp + ll
```

The log-likelihood is literally `-gaussian_nll(...)` – **the same Gaussian NLL equation from Section 5**, not a new statistical model. The unnormalized log-posterior is prior + likelihood, standard Bayes' rule in log-space.

6.6 `run_mcmc_fit` – the sampler

Function: `run_mcmc_fit` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 623-765` **Walker initialization** (small Gaussian ball around the existing `curve_fit` solution θ_0):

$$\theta_0^{(w)} = \theta_0 + \epsilon \cdot \mathcal{N}(0,1), \qquad \epsilon = \max(10^{-3}|\theta_0|,\ 10^{-4})$$

Posterior summary (median + 16th/84th percentile, i.e. asymmetric 1σ-equivalent errors):

$$\hat{\theta}_k = \mathrm{P}_{50}(\theta_k), \qquad \sigma_k^- = \hat{\theta}_k - \mathrm{P}_{16}(\theta_k), \qquad \sigma_k^+ = \mathrm{P}_{84}(\theta_k) - \hat{\theta}_k$$

Convergence check (autocorrelation time τ):

$$\text{converged} \iff N_{\text{steps}} > 50 \max_k(\tau_k)$$

```
def run_mcmc_fit(t, y, yerr, model_type, period_mode, popt_dict, *,
                 P1=None, P2=None, P1_err_est=None, P2_err_est=None,
                 minP=1.0, maxP=200.0, period_prior_k=3.0,
                 nwalkers=64, nsteps=5000, nburn=1000, thin=5,
                 seed=None, progress=False, moves="de"):
    if not EMCEE_OK:
        raise RuntimeError("emcee is not installed; pip install emcee")

    spec = MCMC_PARAM_SPECS[(model_type, period_mode)]
    names = [s[0] for s in spec]
    ndim = len(spec)
    rng = np.random.default_rng(seed)

    fixed = {}
    if period_mode == "fixed":
        if model_type == "twin":
            fixed["P1"] = P1; fixed["P2"] = P2
        else:
            fixed["P"] = P1

    amp_scale = float(np.nanmax(np.abs(y - np.nanmedian(y))))
    if not np.isfinite(amp_scale) or amp_scale <= 0:
        amp_scale = 1.0
    amp_bounds = (0.0, 8.0 * amp_scale)
    const_bounds = (float(np.nanmin(y)) - 2 * amp_scale, float(np.nanmax(y)) + 2 * amp_scale)

    period_priors = {}
    if period_mode == "free":
        def _sigma_for(P0, Perr):
            if Perr is not None and np.isfinite(Perr) and Perr > 0:
                return period_prior_k * Perr
            return max(period_prior_k * 0.01 * P0, 1e-6)
        if model_type == "single":
            period_priors["P"] = (P1, _sigma_for(P1, P1_err_est), minP, maxP)
        else:
            period_priors["P1"] = (P1, _sigma_for(P1, P1_err_est), minP, maxP)
            period_priors["P2"] = (P2, _sigma_for(P2, P2_err_est), minP, maxP)

    prior_cfg = dict(amp_bounds=amp_bounds, const_bounds=const_bounds, period_priors=period_priors)

    theta0 = []
    for name, kind in spec:
```

```
        if name in popt_dict:
            v = popt_dict[name]
        elif name in ("P", "P1"):
            v = P1
        elif name == "P2":
            v = P2
        else:
            raise KeyError(f"Missing starting value for MCMC parameter '{name}'")
        if kind == "phase":
            v = float(_wrap_phase(np.array([v]))[0])
        if kind == "amp":
            v = abs(v)
        theta0.append(v)
    theta0 = np.array(theta0, float)

    step_scale = np.maximum(np.abs(theta0) * 1e-3, 1e-4)
    p0 = []
    tries = 0
    while len(p0) < nwalkers and tries < nwalkers * 200:
        cand = theta0 + step_scale * rng.normal(size=ndim)
        if np.isfinite(_mcmc_log_prior(cand, spec, prior_cfg)):
            p0.append(cand)
        tries += 1
    scale_mult = 10.0
    while len(p0) < nwalkers:
        cand = theta0 + scale_mult * step_scale * rng.normal(size=ndim)
        if np.isfinite(_mcmc_log_prior(cand, spec, prior_cfg)):
            p0.append(cand)
        scale_mult *= 1.5
    p0 = np.array(p0[:nwalkers])

    if moves == "de":
        move_strategy = [(emcee.moves.DEMove(), 0.8), (emcee.moves.DESnookerMove(), 0.2)]
    else:
        move_strategy = None

    sampler = emcee.EnsembleSampler(nwalkers, ndim, _mcmc_log_posterior,
                                    args=(spec, fixed, prior_cfg, t, y, yerr),
                                    moves=move_strategy)
    sampler.run_mcmc(p0, nsteps, progress=progress)

    acc_frac = float(np.mean(sampler.acceptance_fraction))
    try:
        tau = sampler.get_autocorr_time(quiet=True)
        tau_max = float(np.nanmax(tau))
        converged = bool(nsteps > 50 * tau_max)
    except Exception:
        tau_max = float("nan"); converged = False

    nburn_eff = min(nburn, max(nsteps - 1, 0))
    flat = sampler.get_chain(discard=nburn_eff, thin=max(1, thin), flat=True)
    chain = sampler.get_chain()

    flat_report = flat.copy()
    for i, (name, kind) in enumerate(spec):
        if kind == "phase":
            flat_report[:, i] = _wrap_phase(flat[:, i])

    summary = {}
    for i, (name, kind) in enumerate(spec):
        p16, p50, p84 = np.percentile(flat_report[:, i], [16, 50, 84])
        summary[name] = dict(median=float(p50), err_lo=float(p50 - p16), err_hi=float(p84 - p50))

    return dict(
        model_type=model_type, period_mode=period_mode, param_names=names, spec=spec,
        summary=summary, flat_samples=flat_report, chain=chain,
        acceptance_fraction=acc_frac, autocorr_time_max=tau_max, converged=converged,
        nwalkers=nwalkers, nsteps=nsteps, nburn=nburn_eff, thin=thin,
```

```
    )
```

Key design choices: (i) periods are held fixed *or* free per Section 6.5's prior; (ii) the sampler defaults to a `DEMove+DESnookerMove` mixture (`emcee`'s own recommended strategy for correlated/degenerate posteriors – amplitude, phase, and period are typically correlated here), which empirically gives a much lower autocorrelation time τ than the plain stretch move on real, noisy, multi-observer photometry; (iii) walkers failing the prior are resampled, widening the ball if needed.

6.7 `make_mcmc_plots`

Function: `make_mcmc_plots` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 767-795`

```
def make_mcmc_plots(outdir, tag, result):
    names = result["param_names"]
    flat = result["flat_samples"]
    chain = result["chain"]
    if CORNER_OK:
        fig = corner_pkg.corner(flat, labels=names, quantiles=[0.16, 0.5, 0.84],
                                show_titles=True, title_fmt=".4g")
        fig_save_all_ext(fig, os.path.join(outdir, f"09_mcmc_corner_{tag}.png"), dpi_png=150)
        plt.close(fig)
    ndim = len(names)
    fig, axes = plt.subplots(ndim, 1, figsize=(9, 1.8 * ndim), sharex=True, squeeze=False)
    axes = axes[:, 0]
    for i in range(ndim):
        axes[i].plot(chain[:, :, i], alpha=0.4, lw=0.5)
        axes[i].set_ylabel(names[i])
        axes[i].axvline(result["nburn"], color="k", ls="--", lw=0.8)
    axes[-1].set_xlabel("Step (dashed line = end of burn-in)")
    fig.suptitle(f"MCMC trace: {tag}")
    fig.tight_layout()
    fig_save_all_ext(fig, os.path.join(outdir, f"10_mcmc_trace_{tag}.png"), dpi_png=150)
    plt.close(fig)
```

Saves the posterior corner plot (pairwise 2D marginal distributions with 16/50/84th percentile quantile lines) and the per-walker trace plot (parameter value vs. step, burn-in marked with a dashed line) for visual convergence inspection.

6.8 `run_all_mcmc` – top-level orchestrator

Function: `run_all_mcmc` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, lines 797-836`

```
def run_all_mcmc(outdir, t, y, yerr, params, P1, P2, minP, maxP, *,
                 nwalkers=64, nsteps=5000, nburn=1000, thin=5,
                 period_prior_k=3.0, seed=None, progress=False, moves="de"):
    (A1_single, phi1_single, C_single), _ = fit_weighted_sine(t, y, yerr, P1)

    out = {}
    combos = [("single", "fixed"), ("single", "free"), ("twin", "fixed"), ("twin", "free")]
    for model_type, period_mode in combos:
        tag = f"{model_type}_{period_mode}"
        if model_type == "single":
            popt_dict = dict(A=A1_single, phi=phi1_single, C=C_single)
        else:
            popt_dict = dict(A1=params["A1"], phi1=params["phi1"],
                             A2=params["A2"], phi2=params["phi2"], C=params["C"])
        res = run_mcmc_fit(
            t, y, yerr, model_type, period_mode, popt_dict,
            P1=P1, P2=P2, P1_err_est=params.get("P1_err_est"), P2_err_est=params.get("P2_err_est"),
            minP=minP, maxP=maxP, period_prior_k=period_prior_k,
            nwalkers=nwalkers, nsteps=nsteps, nburn=nburn, thin=thin,
            seed=seed, progress=progress, moves=moves,
        )
```

```
        make_mcmc_plots(outdir, tag, res)
        out[tag] = dict(
            model_type=model_type, period_mode=period_mode, summary=res["summary"],
            acceptance_fraction=res["acceptance_fraction"], autocorr_time_max=res["autocorr_time_max"],
            converged=res["converged"], nwalkers=res["nwalkers"], nsteps=res["nsteps"],
            nburn=res["nburn"], thin=res["thin"],
        )
    return out
```

Runs all four (model, period-mode) combinations, re-fitting the single-sine model independently (via the existing `fit_weighted_sine`) only to obtain its walker starting point. Returns a JSON-serializable dict (samples/chains are deliberately excluded from the return value to keep `summary.json` small – only scalar posterior summaries and diagnostics are kept).

7 Pipeline Orchestration: `main()`

Function: `main` **File:** `phot_var_twin_sine_QC_v12_mcmc.py, line 1085`

`main()` parses ∼45 CLI arguments (grouped: I/O & columns, subsetting, cleaning/QC, period search & priors, MCMC, meta/output) and executes the chronological sequence from Section 1:

```
df_in = read_table(args.input, delimiter=delim, comment=args.comment_char, skiprows=args.skiprows)
colmap = auto_map_columns(df_in, user_map=user_map)
df = prepare_data(df_in, colmap, normalize=True)
...
df_clean, removed = smart_clean(df, ...)
...
d_qc = tag_sessions(df_clean, group=args.group, gap_hours=args.gap_hours).copy()
sessions = summarize_sessions(d_qc)
bad_mask = flag_bad_sessions(sessions, args.qc_chi_thresh, args.qc_ofrac_thresh, args.qc_minN, args.
    qc_mad_factor)
obs = rollup_observers(sessions)
...
df_clean = tag_sessions(df_clean, group=args.group, gap_hours=args.gap_hours)
...
(Pgrid, power, P1, resid1, Ph, pow_h, Pd, pow_d, P2, params,
 model_single, metrics_single, model_twin, metrics_twin) = prewhiten_twin_sine(...)
...
if args.mcmc:
    mcmc_results = run_all_mcmc(...)
...
make_plots(outdir, df, df_clean2, removed if len(removed)>0 else None, ...)
```

It then writes `summary.json` (all parameters, metrics, QC counts, and – if `-mcmc` was used – the `mcmc` sub-dict), `fit_report.txt` (human-readable version, with an MCMC section appended when applicable), and the full set of CSV/PNG/PDF outputs referenced throughout this document.

8 Comparison / Overleaf Reporting Script

Function: `(all functions below)` **File:** `v10_compare_runs_to_overleaf_mcmc.py` This script performs **no new statistical modeling** – it reads one or more output directories produced by `main()` above and assembles a LaTeX/PDF comparison report.

8.1 String/formatting utilities

Function: `tex_escape, sanitize_for_filename, safe_read_json, safe_read_csv_rows, fmt_num, fmt_int, wrap_label` **File:** `lines 10-65`

```
def tex_escape(s: str) -> str:
    if s is None: return ""
    repl = {
        "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#",
        "_": r"\_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}",
        "^": r"\textasciicircum{}", "\\": r"\textbackslash{}",
    }
    return "".join(repl.get(ch, ch) for ch in str(s))

def fmt_num(v, spec):
    try:
        if v is None: return ""
        f = float(v)
        if f != f: return "" # NaN
        return format(f, spec)
    except Exception:
        return ""

def wrap_label(label: str, maxlen: int = 28) -> str:
    """Insert LaTeX line breaks to avoid overfull columns."""
    words = str(label).split()
    lines, cur = [], ""
    for w in words:
        nxt = (cur + " " + w).strip()
        if len(nxt) <= maxlen:
            cur = nxt
        else:
            if cur: lines.append(cur)
            cur = w
    if cur: lines.append(cur)
    return r"\begin{tabular}[t]{@{}l@{}}" + r" \\ ".join(tex_escape(x) for x in lines) + r"\end{tabular}"
```

No mathematics – LaTeX-safety escaping, filename sanitizing, safe JSON/CSV reads that degrade to empty/zero rather than crashing, and numeric formatting helpers.

8.2 `collect_cleaning_breakdowns` – removed-fraction

Function: `collect_cleaning_breakdowns` **File:** `lines 68-116`

$$\text{rem\_frac} = \frac{\text{removed}}{\text{kept} + \text{removed}}$$

```
tmp["kept"] = tmp.get("kept", 0).fillna(0).astype(float)
tmp["removed"] = tmp.get("removed", 0).fillna(0).astype(float)
denom = tmp["kept"] + tmp["removed"]
tmp["rem_frac"] = tmp["removed"] / denom.where(denom > 0, pd.NA)
tmp_sorted = tmp.sort_values(["rem_frac", "removed", "kept"], ascending=[False, False, True])
```

Reads `cleaning_summary_by_observer.csv` / `cleaning_summary_by_session.csv` and computes the removed-fraction per observer, used only to *sort* the worst-behaved observers to the top of the comparison table – a simple ratio, not a new statistical estimator.

8.3 `collect_run` – pulls everything from `summary.json`

Function: `collect_run` **File:** `lines 118-233`

```
sj = run_dir / "summary.json"
summ = safe_read_json(sj)
if summ:
    r["n_input"] = summ.get("n_points_input")
    ...
    per = summ.get("periods", {})
```

```
    r["P1"] = per.get("P1"); r["P1_err"] = per.get("P1_err_est")
    r["P2"] = per.get("P2"); r["P2_err"] = per.get("P2_err_est")
    met = summ.get("metrics", {})
    r["met_single"] = met.get("single_sine", {}) or {}
    r["met_twin"] = met.get("twin_sine", {}) or {}
    ...
    # NEW: MCMC posterior results, if the run was made with --mcmc (else None)
    r["mcmc"] = summ.get("mcmc")
```

Every number here (P_1, P_2, χ^2, AIC, BIC, MCMC summaries, QC counts) is read verbatim from the analysis script's `summary.json` – i.e. it was already computed by the equations in Sections 2–6 above and is only being copied into this script's internal dict for table-building.

8.4 Figure discovery: `FIG_BASES`, `_find_best_fig`, `copy_and_rename_figs`

Function: `FIG_BASES, _find_best_fig, copy_and_rename_figs` **File:** `lines 237-287`

```
FIG_BASES = {
    "06_phase_P1": "Phase-folded at P1",
    "05_residuals": "Residuals vs. time",
    "04_model_timeseries": "Model vs. time",
    "09_mcmc_corner_twin_free": "MCMC corner (twin-sine, free period)",
    "10_mcmc_trace_twin_free": "MCMC trace (twin-sine, free period)",
}
PREF_EXT = [".pdf", ".PDF", ".png", ".PNG"]
```

No mathematics – locates the best-available (PDF preferred over PNG) copy of each named figure per run directory and copies it into the report's `figs/` folder under a sanitized, per-run filename. Missing files (e.g. MCMC figures for a run made without `-mcmc`) are skipped gracefully, not treated as errors.

8.5 `build_tex` – assembling the report

Function: `build_tex` **File:** `lines 289-539`

$$\text{keep}\% = 100 \cdot \frac{N_{\text{clean}}}{N_{\text{input}}}$$

```
keep = f"{100.0*float(r['n_clean'])/float(r['n_input']):.1f}"
```

Beyond this one ratio, `build_tex` contains no new mathematics – it assembles `longtable`/`tabularx` LaTeX blocks for: per-run settings, cleaning breakdowns, model metrics (χ^2, AIC, BIC for both models), the MCMC posterior-summary table (twin-sine, both period modes, with acceptance fraction and convergence flag pulled straight from `run_all_mcmc`'s output), and the figure blocks from the previous subsection. When no run has MCMC data, that entire section is omitted; when only some runs do, missing ones render as "(no MCMC data)" rather than breaking the table.

8.6 `main` (reporting script)

Function: `main` **File:** `line 541` Parses positional run directories + `-labels`/`-outdir`, calls `collect_run()` on each, then `build_tex()`, and writes the final `comparison_report.tex` (compilable standalone to PDF).

9 Complete Function Index

Function	File	Lines	Section
fig_save_all_ext	script 1	49–53	Utilities
plt_save_all_ext	script 1	55–59	Utilities
timestamp	script 1	61–62	Utilities
ensure_dir	script 1	64–65	Utilities
mad	script 1	69–71	Utilities (MAD)
weighted_stats	script 1	73–79	Utilities
read_table	script 1	93–108	Data Ingestion
auto_map_columns	script 1	110–130	Data Ingestion
prepare_data	script 1	132–170	Data Ingestion (mag→flux)
parse_exclude_periods	script 1	1070–1083	Utilities (CLI)
tag_sessions	script 1	173–198	Quality Control
per_group_metrics	script 1	200–227	Quality Control
summarize_sessions	script 1	229–241	Quality Control
rollup_observers	script 1	243–252	Quality Control
flag_bad_sessions	script 1	254–265	Quality Control
_barh	script 1	267–281	Quality Control (plotting)
smart_clean	script 1	284–403	Quality Control
gls_periodogram	script 1	406–418	GLS Period Search
_safe_gls_window	script 1	420–423	GLS Period Search
apply_exclude_notches	script 1	425–432	GLS Period Search
sine_model	script 1	434–435	Single/Twin-Sine Model
twin_sine_model	script 1	437–438	Single/Twin-Sine Model
fit_weighted_sine	script 1	440–448	Single/Twin-Sine Model
fit_twin_sine	script 1	450–458	Single/Twin-Sine Model
gaussian_nll	script 1	460–467	Single/Twin-Sine Model (also reused by MCMC)
chi2_metrics	script 1	469–524	Single/Twin-Sine Model
MCMC_PARAM_SPECS	script 1	549–555	MCMC
_mcmc_build_model	script 1	557–570	MCMC
_wrap_phase	script 1	572–575	MCMC
_mcmc_log_prior	script 1	577–603	MCMC
_mcmc_log_likelihood	script 1	605–612	MCMC
_mcmc_log_posterior	script 1	614–621	MCMC
run_mcmc_fit	script 1	623–765	MCMC
make_mcmc_plots	script 1	767–795	MCMC
run_all_mcmc	script 1	797–836	MCMC
prewhiten_twin_sine	script 1	838–964	GLS Period Search *and* Single/Twin-Sine Model (spans both)
estimate_period_err (nested)	script 1	908–947	GLS Period Search

Function	File	Lines	Section
make_plots	script 1	967–1067	Single/Twin-Sine Model (phase folding)
main	script 1	1085–end	Orchestration
tex_escape	script 2	10–17	Reporting
sanitize_for_filename	script 2	19–20	Reporting
safe_read_json	script 2	22–26	Reporting
safe_read_csv_rows	script 2	28–32	Reporting
fmt_num	script 2	34–42	Reporting
fmt_int	script 2	44–51	Reporting
wrap_label	script 2	53–65	Reporting
collect_cleaning_breakdowns	script 2	68–116	Reporting (rem_frac)
collect_run	script 2	118–233	Reporting
_find_best_fig	script 2	248–268	Reporting
copy_and_rename_figs	script 2	270–287	Reporting
build_tex	script 2	289–539	Reporting (keep%)
main	script 2	541–end	Reporting Orchestration

Legend: "script 1" = phot_var_twin_sine_QC_v12_mcmc.py; "script 2" = v10_compare_runs_to_overleaf_mcn